



Dreamer's Journey: From PlaNet to V4

Ahmet S. Asarkaya

Agenda

1

Introduction

What is it all about?

2

Dreamer Family

What is the story?

3

DreamerV4

SOTA World Model

4

Discussion

Questions?



1

Introduction

What is it all about?

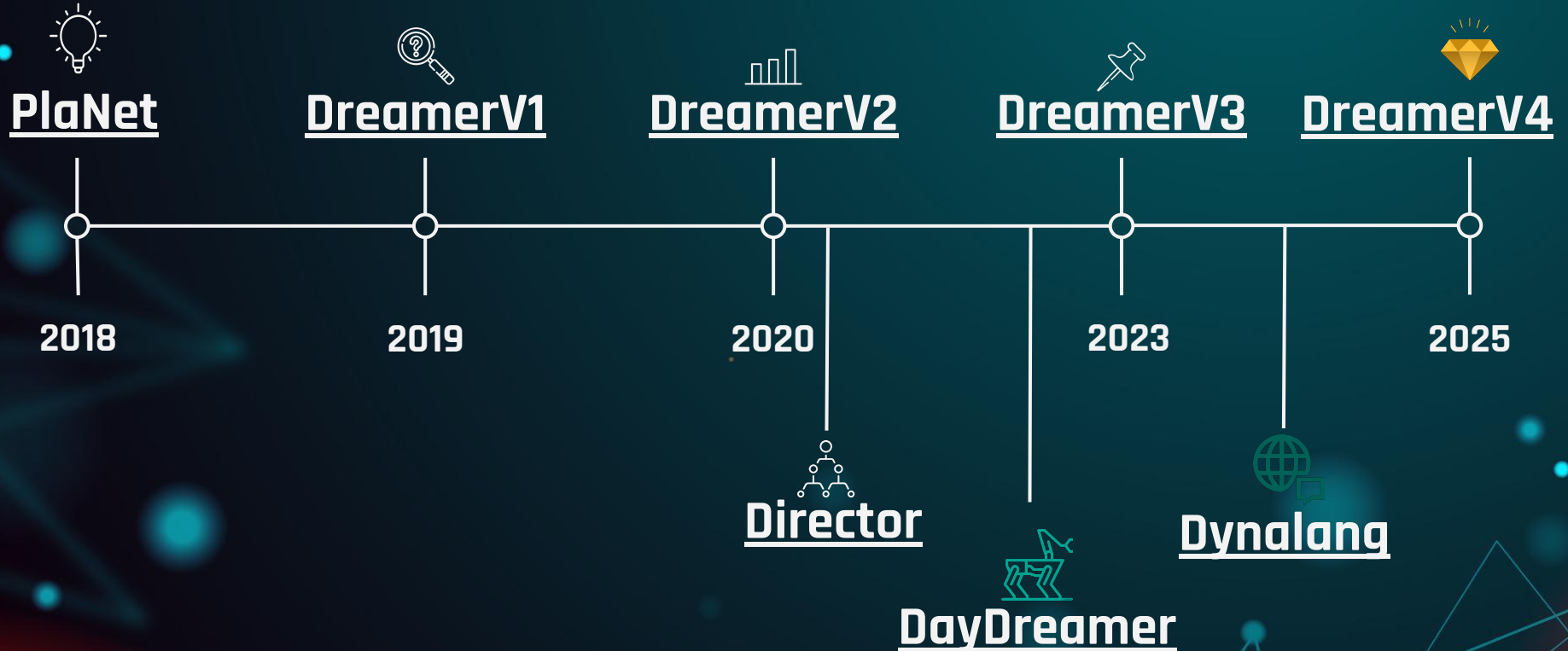
While *world models* as a concept is not new, **World Models** (Ha & Schmidhuber, 2018) used "**dream**" as a technical term in RL. The **Dreamer series** (Hafner, Danijar, et al.) then turned it into a global research vocabulary.

- +7000 citations
- From a single metaphor introduced in 2018 to over dozens of papers literally named after the dream concept – spanning RL, robotics, 3D generation, compilers, software, navigation, driving, and more (**DreamZero**, **DreamDojo**, **DriveDreamer**, **Dreamwalker**, **Dreamgen**, ...)

A word cloud of terms related to the 'Dreamer' concept in reinforcement learning. The words are arranged in a roughly rectangular shape, with some terms appearing more frequently or in larger fonts than others. The terms include:

- WorldDreamer
- TransDreamer
- Dreamshard
- Harmonydream
- Layout-aware dreamer
- Dream to Recall
- Optimizing bucket-filling... ... inside a dream environment
- Dreamsmooth
- SkyDreamer
- Pedestrian scenarios using a dreamer-based motion planner
- Agent planning with dreaming
- Dream to manipulate
- Flowdreamer
- Iso-dreamer
- Dreaming
- DreamTrack: Dreaming the Future
- Dream to drive
- Dream to Recall
- Dream to generalize
- Dream to adapt
- When World Models Dream Wrong
- Strategic dreaming
- CarDreamer
- Dreamshard
- Dream to fly
- Dreamernav
- Compilerdream
- Meddreamer
- Dreaming to assist

Lineage of Dreamer Family



Key Ideas and Concepts

What is a World Model in this series?

- A world model is a learned internal model of the environment that **predicts future states** and **rewards** based on **past observations** and **future actions**. It lets the agent “**imagine**” (**dream**) interactions

How does it work?

- Encode high-dimensional inputs (e.g., pixels) into a **latent state space**
- Learn **dynamics** in this latent space

How is it used in this series?

- The learned WM is used for **planning** or **policy learning** without needing real interactions for every update

THE THREE PILLARS OF DREAMER



Learn the World

Compress high-dimensional observations (pixels) into a compact latent state space.



Dream the Future

Use the learned model to roll out trajectories in imagination..



Optimize Policy

Train a policy within the "dream" to maximize expected returns.



2

Dreamer Family

What is the story?

Models	What is new?	World Modeling	Policy Learning	Benchmarks
PlaNet	- Planning in latent space - Recurrent State Space Model (RSSM) - Latent Overshooting	- Encoder/Decoder (CNNs) - Dynamic Model: RSSM (GRU) - Reward Function (MLP)	- Cross Entropy Method (CEM) - Model Predictive Control (MPC)	DM Control Suite
DreamerV1	- Policy learning via actor-critic in imagination	- Encoder/Decoder (CNNs) - Dynamic Model: RSSM (GRU) - Reward Function (MLP)	- Soft Actor Critic (SAC) - Actor Model (MLP) - Critic Model (MLP)	Atari, DMLab
DreamerV2	- Categorical latent states in the WM - Modified WM loss function - Increase in model size - Regularized policy pipeline	- Encoder/Decoder (CNNs) - Dynamic Model: RSSM (GRU) - Reward Function (MLP)	- Soft Actor Critic (SAC) - Actor Model (MLP) - Critic Model (MLP)	Atari
DreamerV3	- SOTA performance across many domains with a fixed set of hyperparameters - First agent to collect Diamond in Minecraft via pure RL	- Encoder/Decoder (CNNs) - Dynamic Model: RSSM (GRU) - Reward Function (MLP)	- Soft Actor Critic (SAC) - Actor Model (MLP) - Critic Model (MLP)	Atari, Procgen, DMLab, BSuite, Crafter, Minecraft
DreamerV4	- A high-capacity world model achieving real-time inference on a single GPU - First agent to collect Diamond in Minecraft from an offline dataset - WM can learn from both unlabeled and labeled videos	- Encoder/Decoder (Transf.) - Dynamic Model: RSSM (DiT) - Reward Function (MLP)	- PMPO - Actor Model (MLP) - Critic Model (MLP)	Minecraft

One Sentence Summaries

PlaNet



Leveraged the **idea** of planning in latent space **without** pixel-level **reconstruction**

DreamerV1



Replaced PlaNet's **search-based** planning with an **actor-critic** approach

DreamerV2



Leveraged **discrete** (categorical) latent states instead of **Gaussian** latent variables

Director



Proposed a **hierarchical** learning for efficient long-horizon decision-making

Daydreamer



Demonstrated Dreamer's ability to **learn in the real world**

DreamerV3



Unified training across diverse domains with **fixed** hyperparameters and larger models

Dynalang



Incorporated **language** to enable multimodal world modeling

DreamerV4



Replacing RNNs with **diffusion transformers** – obtaining **diamond** in Minecraft via training in pure **imagination**

Minecraft Dataset - Highlights

- **2541** hours of contractor gameplay @**20FPS** (~182M frames):

- Observations: **360x640** RGB images
 - Zero padded to 384x640
- **Actions:**
 - **Keyboard:** 23 binary actions
 - **Mouse:** discretized into 121 classes
- **90%** training | **10%** validation split

Index	Task
1	mine_log
2	mine_cobblestone
3	mine_iron_ore
4	mine_coal
5	mine_diamond
6	craft_planks
7	craft_stick
8	craft_crafting_table
9	craft_furnace
10	craft_iron_ingot
11	craft_wooden_pickaxe
12	craft_stone_pickaxe
13	craft_iron_pickaxe
14	open_crafting_table
15	open_furnace
16	place_crafting_table
17	place_furnace
18	use_wooden_pickaxe
19	use_stone_pickaxe
20	use_iron_pickaxe

Table 4: Taskset for training the multi-task agent.

- **Recall that:** Dreamer V3 learns to collect diamond from scratch **with 1.4k hours of online interactions** (with NO human data) in a simplified Minecraft environment
 - Abstract crafting (e.g., holding space bar, ...)
 - Low-resolution
 - Encoded inventory

Agent	Inputs	Actions	Data (hours)		
			Offline	Web	Online
Dreamer 3	64x64, inventory	keyboard, camera, abstract crafting	—	—	1.4K
VPT (RL)	128x128	keyboard, mouse	2.5K	270K	194K
VPT (BC)	128x128	keyboard, mouse	2.5K	270K	—
Dreamer 4	360x640	keyboard, mouse	2.5K	—	—

20 unique subtasks need
to be solved to reach
diamond; total of **~20 mins**
on avg (**~24k actions**)

Task	Count
mine_log	10
craft_planks	20
craft_crafting_table	1
place_crafting_table	1
craft_stick	4
craft_wooden_pickaxe	1
use_wooden_pickaxe	1
mine_cobblestone	3
craft_planks	4
craft_crafting_table	1
place_crafting_table	1
craft_stick	4
craft_stone_pickaxe	1
use_stone_pickaxe	1
mine_iron_ore	3
mine_cobblestone	8
craft_planks	4
craft_crafting_table	1
place_crafting_table	1
craft_furnace	1
craft_iron_ingot	3
craft_stick	2
craft_iron_pickaxe	1
use_iron_pickaxe	1
mine_diamond	∞

Table 6: Prompt sequence for evaluation.



3

DreamerV4

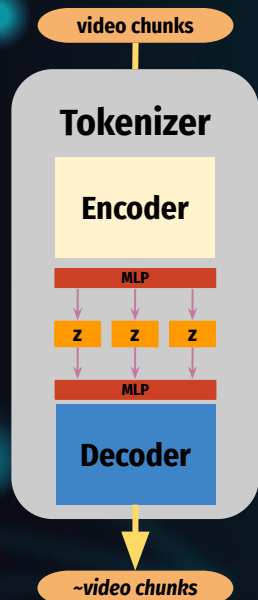
SOTA World Model

Training Agents Inside of Scalable World Models

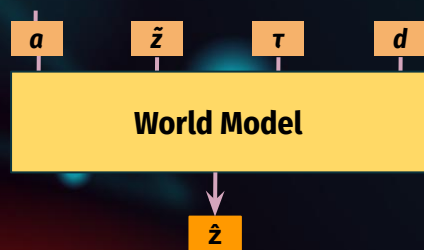


Phase 1

Tokenizer training

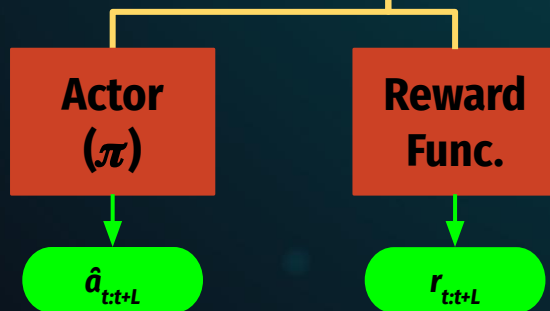
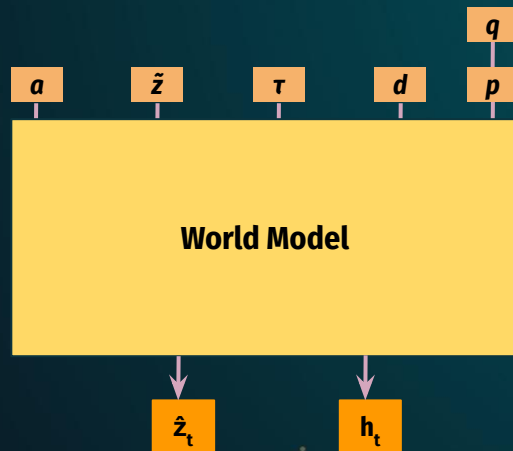


WM pretraining



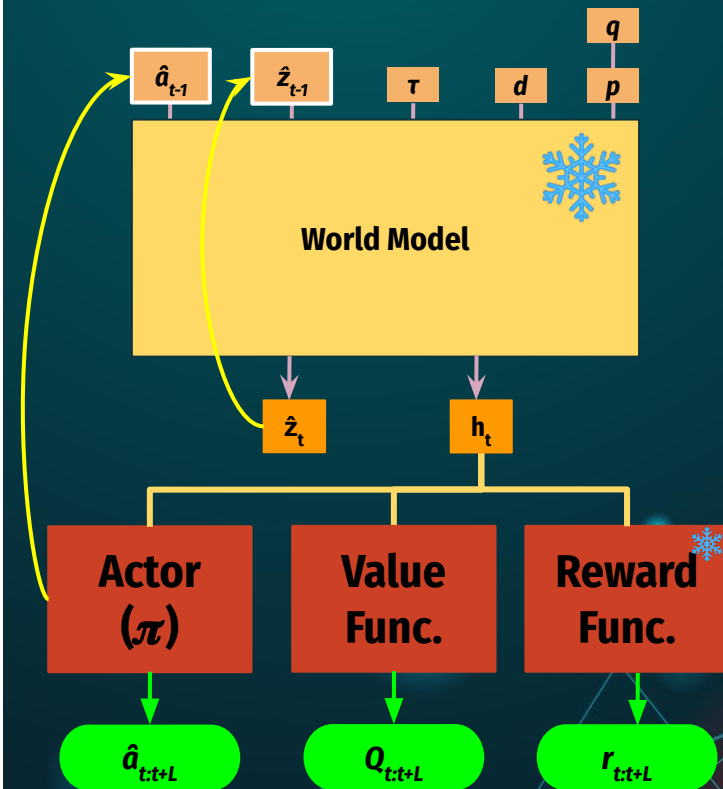
Phase 2

- WM finetuning/adapting
- Actor training via BC
- Reward predictor training



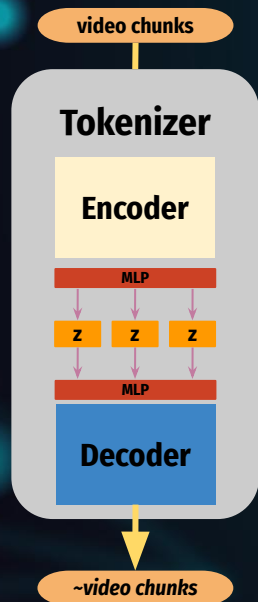
Phase 3

- Actor training via RL
- Value function training

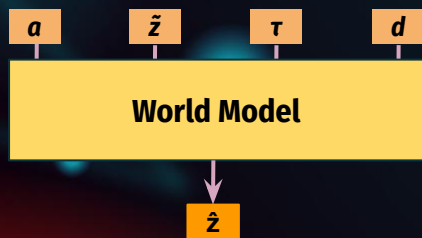


Phase 1

Tokenizer training

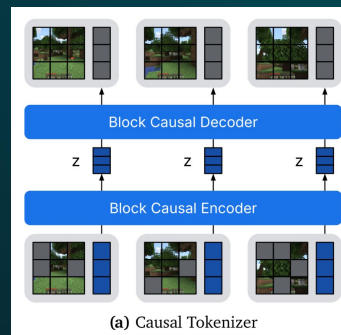


WM pretraining



Tokenizer

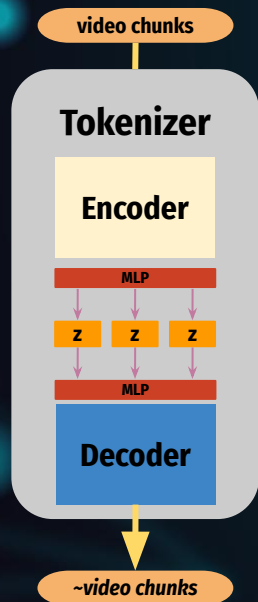
- Encoder/decoder transformers: **400M** params
- Bottleneck, z:**
 - 384x640x3** -> **16x16** patches (**960** tokens) -> **512x16**
 - Compression ratio: **~84**
- Masked autoencoding (MAE): **dropout** the **patches** with $p \sim U(0, 0.9)$



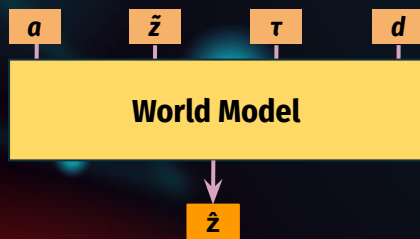
- τ**: Signal level $[0, 1]$ -> 1 means clean data; -> 0 means full noise
- d**: Step size -> how many times to denoise to get from corrupted to clean representations

Phase 1

Tokenizer training

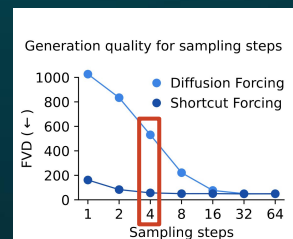


WM pretraining



Dynamic Model (WM)

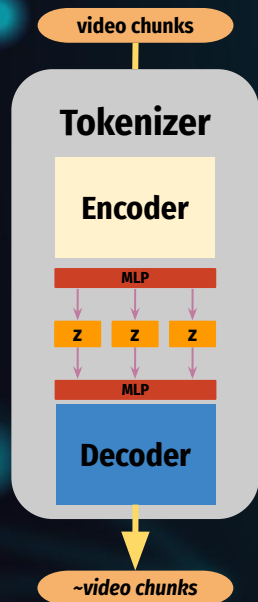
- Diffusion transformer (1.6B) with **shortcut forcing** objective
 - Diffusion forcing**: Different signal level at each time step of the data sequence
 - Inference time**: Supports generating the next frame given clean or lightly noised history ($\tau_{\text{ctx}}=0.1$)
 - Shortcut models**: Condition the neural network not only on the noise level but also the **desired step size**
 - Self-consistency targets**: one shortcut step equals two consecutive shortcut steps of half the size
 - K=4



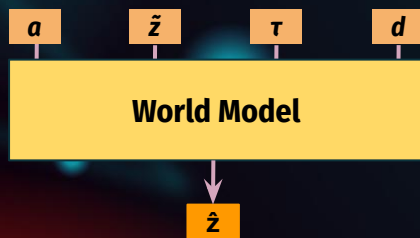
- Random** sequence of sub-tasks are uniformly sampled for training the WM
 - To avoid **optimistic** generation (dreaming)

Phase 1

Tokenizer training



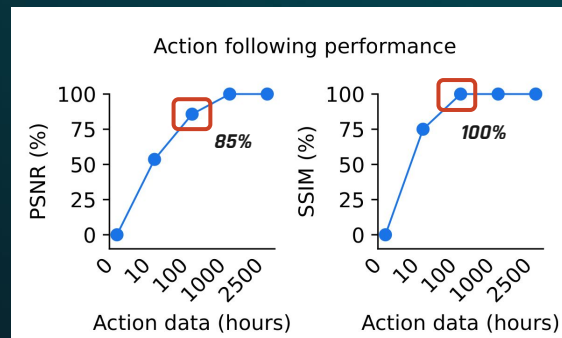
WM pretraining



!!! Used to be online dataset !!!

Dynamic Model (WM)

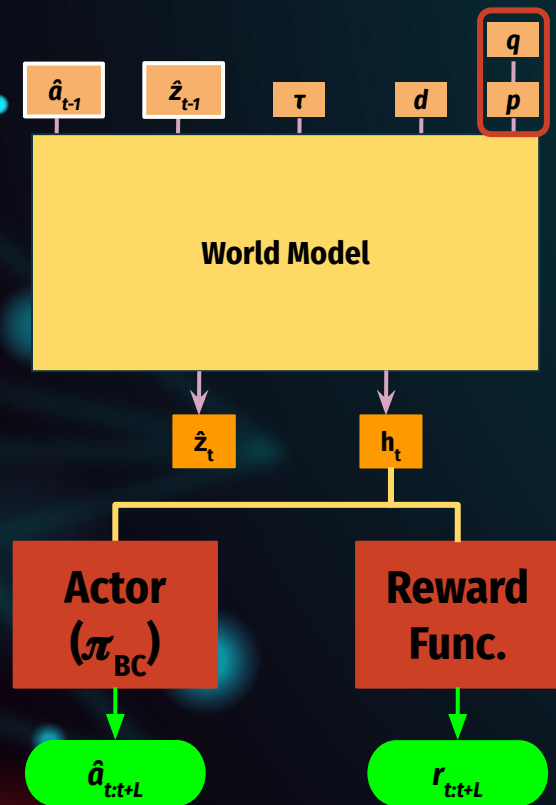
- Training via both labeled and unlabeled videos (**completely offline dataset**)
 - Raw actions $\rightarrow$ action embeddings ($\mathbf{S}_{a_mouse}$, $\mathbf{S}_{a_keyboard}$)
- When using **unlabeled** videos $\mathbf{S}_a = \mathbf{S}_{a_learned}$
 - Learn a **broader** distribution of possible outcomes
 - **Most of knowledge learned from unlabeled videos**
- When using **labeled** videos $\mathbf{S}_a = \mathbf{S}_{a_mouse} + \mathbf{S}_{a_keyboard} + \mathbf{S}_{a_learned}$
 - Learn a **narrow** distribution for the given action labels



!!! Only **100 out of 2500** hrs action labeled data is enough to !!!
!!! achieve **80%** of the action-conditioned generation accuracy !!!

Phase 2

- *WM finetuning/adapting*
- *Actor training via BC*
- *Reward predictor training*



WM finetuning/adapting

- Inserting agent tokens, p
 - Takes **one-hot encoded tasks**, q as input
 - Attend themselves and others
 - **No other modalities attends back** to agent tokens
 - **Avoid causal confusion:**
 - future predictions should only be influenced by the actions not the current task
 - Interleave it with other modalities
- p is used to adapt the pretrained WM to predict actions and rewards

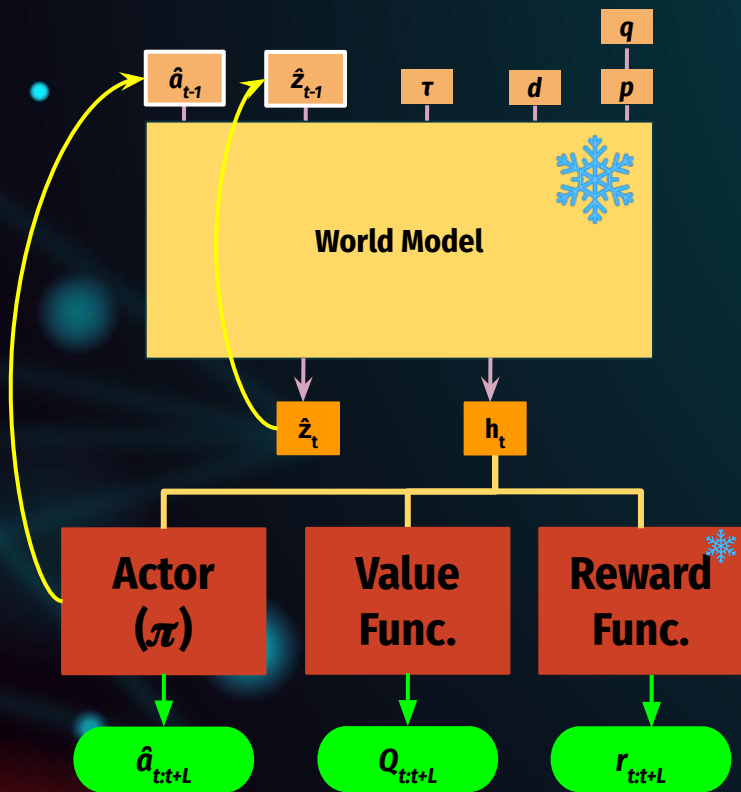
Actor training via BC & Reward predictor training

- Takes task output embedding, h as inputs
- Trained on action chunks of 8 (via multi-token prediction, MTP)
- Small MLPs
 - One output layer per MTP distance

!!! Open loop policy learning via **Imitation Learning** !!!

Phase 3

- Actor training via RL
- Value function training



Actor training via RL

- **Frozen** WM and Reward predictor
 - **NOTE:** Authors mention that full training might provide small benefits at higher computational cost
- The rollouts are generated by **unrolling** the **transformer** with **itself**
 - **Sampling** representations $\hat{z}_t$ from the flow head
 - **Sampling** action $\hat{a}$ from the action head
 - **NOTE:** They use only the first predicted action from the chunk in RL loop
- They use PMPO instead of SAC:
 - **PMPO:** Preference based MPO (Maximum a posteriori Policy Optimisation)

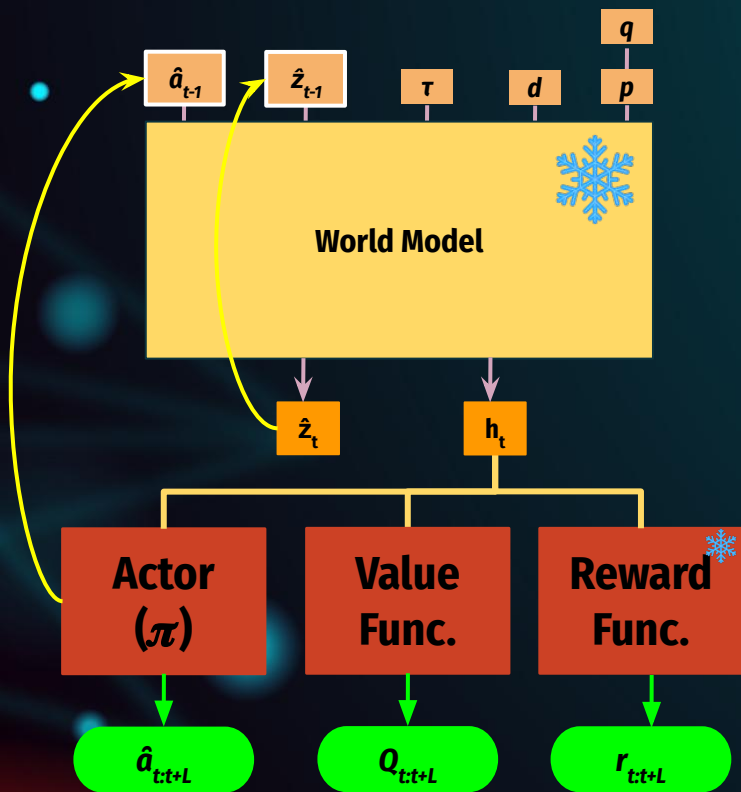
Value function training

- TD-learning

!!! Closed loop policy learning via RL !!!

Phase 3

- Actor training via RL
- Value function training



!!! No offline data is required, **self-sufficient, pure-dreaming**!!!

PMPD

- Uses the **sign** of advantages rather than their magnitude
- Separates **positive-advantage** states ($\mathcal{D}^+$) and **negative-advantage** states ($\mathcal{D}^-$).
 - **Maximizes** log likelihood of actions in $\mathcal{D}^+$
 - **Minimizes** log likelihood of actions in $\mathcal{D}^-$
- Works naturally with negative feedback
- The KL term helps constrain the policy
 - Use the frozen π_{BC} as π_{prior} : Penalizes too dumb actions

$\alpha \rightarrow 0.5$

$\beta \rightarrow 0.3$

$$\mathcal{L}(\theta) = \frac{1 - \alpha}{|\mathcal{D}^-|} \sum_{i \in \mathcal{D}^-} \ln \pi_{\theta}(a_i | s_i) - \frac{\alpha}{|\mathcal{D}^+|} \sum_{i \in \mathcal{D}^+} \ln \pi_{\theta}(a_i | s_i) + \frac{\beta}{N} \sum_{i=1}^N \text{KL}[\pi_{\theta}(a_i | s_i) \parallel \pi_{\text{prior}}]$$

Experiments - Design Choices

Measure the effect of the design choices

- Train each choice for 48 hours
- Measure **FVD** of ~20 seconds windows
- Measure **inference (FPS)** on a single **H100**
- Measure **training iteration speed** on a single **H100**

Model	● Train step seconds	● Inference FPS (↑)	● Quality FVD (↓)
Diffusion Forcing Transformer ●+ Fewer sampling steps ($K = 4$) ●+ Shortcut model ●+ X-Prediction ●+ X-Loss ●+ Ramp weight ●+ Alternating batch lengths ●+ Long context every 4 layers ●+ GQA ●+ Time factorized long context ●+ Register tokens ●+ More spatial tokens ($N_z = 128$) ●+ More spatial tokens ($N_z = 256$)	9.8	0.8	306
		?	

Experiments - Design Choices

Measure the effect of the design choices

- Train each choice for 48 hours
- Measure **FVD** of ~20 seconds windows
- Measure **inference (FPS)** on a single **H100**
- Measure **training iteration speed** on a single **H100**

5.8x faster training
26.8x faster inference
5.4x better video quality

Model	● Train step seconds	● Inference FPS (↑)	● Quality FVD (↓)
Diffusion Forcing Transformer	9.8	0.8	306
●+ Fewer sampling steps ($K = 4$)	9.8	9.1	875
●+ Shortcut model	9.8	9.1	329
●+ X-Prediction	9.8	9.1	326
●+ X-Loss	9.8	9.1	151
●+ Ramp weight	9.8	9.1	102
●+ Alternating batch lengths	1.5	9.1	80
●+ Long context every 4 layers	0.6	18.9	70
●+ GQA	0.5	23.2	71
●+ Time factorized long context	0.4	30.1	91
●+ Register tokens	0.5	28.9	91
●+ More spatial tokens ($N_z = 128$)	0.8	25.7	66
●+ More spatial tokens ($N_z = 256$)	1.7	21.4	57

!!! ~11x increase in FPS !!! Simply reducing sampling steps hurts prediction quality

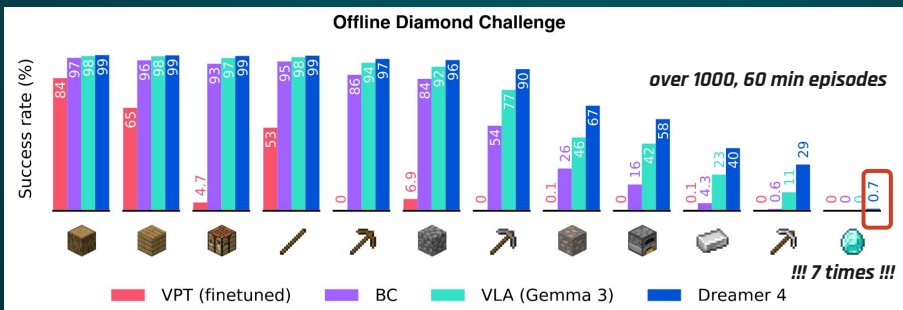
!!! ~2.5x increase in video quality !!! Shortcut model recovers the quality loss

!!! ~7x faster training (and better video quality) !!!

Experiments - Minecraft

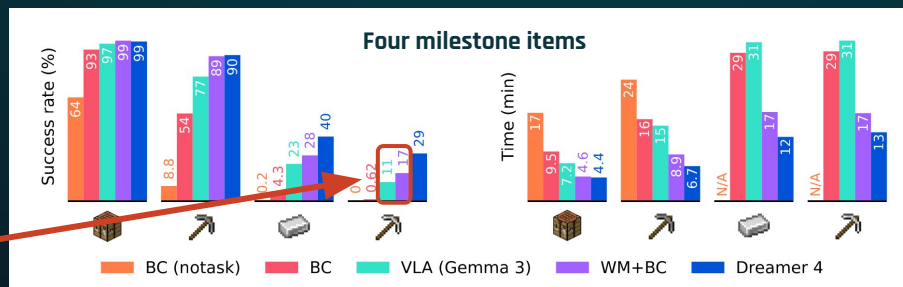
Diamond Collection Challenge

- Using only **raw pixels** as observations
- No online interactions
 - Using a fixed **offline** dataset (VPT)
- **Semi-sparse** reward functions
 - Binary reward for each 20 sub-tasks



Benchmarked Agents (no online interactions)

- **VPT (finetuned)**: 270k hours of YouTube data + finetuning
- **BC (notask)**: Filtered VPT data with no subtask cond.
- **BC**: Filtered VPT data with task cond.
- **VLA**: BC with Gemma 3 as backbone
- **WM+BC**: BC with WM priors (V4 stopped after phase 2)



Experiments - WM Quality

Human Interactions

- Humans try accomplishing tasks
 - Games inside the WM
 - Given the initial frame and instructions
- Compare
 - Inference speed
 - Accuracy of the game mechanics
- Run on a single H100

Benchmarked WMs

- MineWorld
- Lucid-v1
- Oasis

!!! Try yourself !!!

!!! 192 frames, no long term memory of the game state !!!

!!! Fails at '*building*' tasks because of the hallucinations !!!

!!! Generations diverge !!!

Model	Parameters	Resolution	Context	FPS	Success
MineWorld	1.2B	384×224	0.8s	2	0/16
Lucid-v1	1.1B	640×360	1.0s	44	0/16
Oasis (small)	500M	640×360	1.6s	20	5/16
Oasis (large)	—	360×360	1.6s	~5	14/16
Dreamer 4	2B	640×360	9.6s	21	14/16

4

Discussion

Questions?

Unanswered Questions?

- How many **imagined steps** / rollouts were used in imagination training?
- Imagination **rollout length (T)** and bootstrapping details for rollouts
- Compute / **wall-clock details** for the imagination-training phase specifically

What might be next?

- **How can we integrate online interactions into this picture?**
 - How do we efficiently add a stream of environment data to improve and correct the pretrained world model without forgetting? What short corrective online loops make the biggest gains?
- **Long-term memory & episodic recall for longer-horizon planning**
 - Dreamer4's context is still short (seconds). How to add external memory so imagined rollouts can use long histories (days of activity) and plan across them?
- More experiments with real robotic data

Hot Takes 🔥

Simulation is becoming learned, not engineered 👉 Robotics may skip classical sim pipelines

Real robots are data-limited, not compute-limited 👉 Collect once, train forever

Biggest bottleneck is still world model fidelity 👉 We're not yet at reliable real-world deployment

Missing piece: memory + grounding 👉 Next breakthrough = memory-augmented world models

DreamerV4 is less “RL algorithm” and more “robot foundation model (RFM)”

References

- ● Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2019). Learning Latent Dynamics for Planning from Pixels.
- ● Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., McKinney, K., Nguyen, T., Wulfmeier, M., Lee, J., & Davidson, J. (2019). Dream to Control: Learning Behaviors by Latent Imagination.
- ● Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Mastering Atari with Discrete World Models.
- ● Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2023). Mastering Diverse Domains through World Models.
- ● Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Training Agents Inside of Scalable World Models.

!!! Check out my notes to learn more details about these works !!!

QA?